



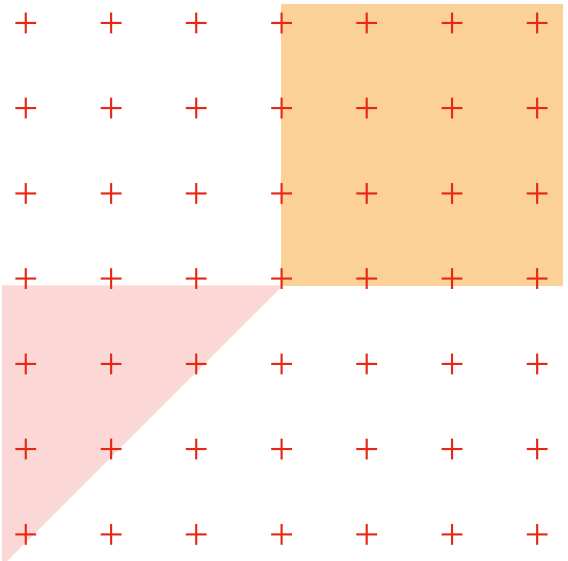
INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
TOULOUSE

RAPPORT PROJET SYSTÈME INFORMATIQUE

Anthony FERRERE-HOAREAU
Titouan MATHIEU

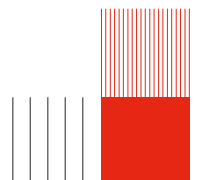
Promotion 60
2025-2026

Compilateur C simplifié
Processeur VHDL
Choix d'implémentation



SOMMAIRE

Introduction	1
1 Choix d'implementation du compilateur	2
1.1 Utilisation de Flex et Bison	2
1.2 Sous-ensemble de C retenu	2
1.3 Table des symboles et gestion de portee	2
1.4 Utilisation de variables temporaires	2
1.5 Choix d'un format d'instruction 32 bits	3
1.6 Generation des sauts	3
2 Choix d'implementation du processeur	4
2.1 Separation en blocs VHDL	4
2.2 Donnees sur 8 bits	4
2.3 ALU	4
2.4 Bancs de donnees	4
2.5 Pipeline minimale	4
2.6 Decodage des bits de poids faible	5
3 Limites et travail restant	6
Conclusion	7

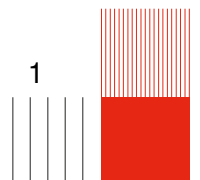


INTRODUCTION

Le projet consiste a relier une partie logicielle, un compilateur pour un sous-ensemble du langage C, a une partie materielle, un processeur decrit en VHDL.

code C simplifie -> instructions 32 bits -> memoire d'instructions -> processeur VHDL

Ce rapport presente les choix d'implementation qui structurent notre projet, les raisons de ces choix, ainsi que les limites actuelles de notre realisation.



1 CHOIX D'IMPLEMENTATION DU COMPILATEUR

1.1 Utilisation de Flex et Bison

Nous avons choisi de separer l'analyse lexicale et l'analyse syntaxique avec Flex et Bison. Ce choix permet de garder une architecture claire : Flex reconnait les elements de base du langage, comme les mots-cles, les identifiants, les nombres et les operateurs, tandis que Bison verifie que ces elements respectent la grammaire du langage.

1.2 Sous-ensemble de C retenu

Le compilateur gere principalement :

- les variables entieres ;
- les affectations ;
- les expressions arithmetiques ;
- les conditions `if/else` ;
- les boucles `while` ;
- l'affichage avec `printf` ;
- le `return`.

Nous avons aussi essaye d'ajouter les appels de fonctions. Cette partie est avancee mais pas totalement finalisee, notamment car elle necessite une convention d'appel plus complete.

1.3 Table des symboles et gestion de portee

La table des symboles associe chaque variable a une adresse logique. Cette adresse est ensuite utilisee dans les instructions generees.

Nous avons choisi une structure de liste chainee enrichie par une information de profondeur. Chaque symbole contient :

- le nom de la variable ;
- son type ;
- sa profondeur de bloc ;
- un lien vers le symbole suivant.

Ce choix se rapproche d'un fonctionnement en pile pour la gestion des blocs. Lorsqu'on entre dans un bloc, la profondeur augmente. Lorsqu'on sort du bloc, les symboles appartenant a cette profondeur sont supprimes. Cela est gere par un indice dans le code Yacc que l'on incremente et decremente.

Ce fonctionnement est adapte a un langage structure par blocs :

```
int a;
if (a == 0) {
    int b;
}
```

Dans cet exemple, `b` ne doit plus etre accessible apres la fermeture du bloc. La profondeur permet donc de respecter la portee des variables.

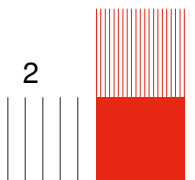
1.4 Utilisation de variables temporaires

Pour compiler les expressions, nous avons choisi de creer des temporaires dans la table des symboles. Cela simplifie la generation du code assembleur.

Par exemple, pour :

```
a = b + c * d;
```

le compilateur peut produire en sortie :



```
MUL tmp1, c, d
ADD tmp2, b, tmp1
COP a, tmp2
```

Chaque operation produit un resultat intermediaire, stocke dans un temporaire, puis reutilisable par l'instruction suivante.

La gestion, l'ajout et la suppression des temporaires dans la table des symboles sont realisees avec un index interne. Cet index memorise la position du dernier symbole non temporaire dans la table. Notre implementation, proche d'une pile, place les temporaires en fin de liste ; il devient donc simple de les supprimer lorsqu'ils ne sont plus utiles en comparant la taille de la liste avec l'index du dernier symbole permanent.

1.5 Choix d'un format d'instruction 32 bits

Nous avons choisi un format fixe de 32 bits :

```
opcode 8 bits | champ1 8 bits | champ2 8 bits | champ3 8 bits
```

Ce format est simple a generer cote compilateur et simple a decoder cote processeur. Il permet d'utiliser la meme structure pour la plupart des instructions :

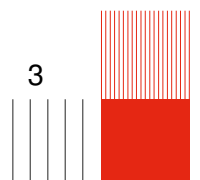
- AFC dst, val : affectation immediate ;
- COP dst, src : copie ;
- ADD dst, a, b : addition ;
- JMP addr : saut ;
- JMF cond, addr : saut conditionnel.

L'inconvenient est que certaines instructions n'utilisent pas les trois champs. Cependant, pour un processeur comme le notre, la simplicite du decodage est plus importante que la compacite du code.

1.6 Generation des sauts

Les structures de controle sont traduites avec des sauts. Pour un `if`, le compilateur genere d'abord la condition, puis un `JMF`. Comme l'adresse de fin du bloc n'est pas connue immediatement, nous avons choisi de placer temporairement une valeur fictive, puis de la modifier apres generation du bloc.

Ce choix impose de conserver les instructions dans une structure modifiable avant affichage. C'est pourquoi les instructions assembleur sont stockees dans une liste chainee avant d'être printer.



2 CHOIX D'IMPLEMENTATION DU PROCESSEUR

2.1 Separation en blocs VHDL

Nous avons separe le processeur en plusieurs composants :

- UAL ;
- banc de registres ;
- memoire d'instructions ;
- memoire de donnees ;
- registres de pipeline.

Ce choix permet de tester chaque bloc separement et de garder une architecture lisible. Par exemple, l'UAL peut etre testee avec un testbench specifique sans lancer toute la pipeline.

2.2 Donnees sur 8 bits

Nous avons choisi de manipuler des donnees sur 8 bits. Ce choix limite les valeurs manipulables, mais il simplifie fortement :

- les tests ;
- l'affichage des resultats ;
- la taille du banc de registres ;
- la lecture des simulations.

Pour ce projet, l'objectif etait de valider le chemin de donnees et le controle, pas de maximiser la taille des entiers.

2.3 ALU

Afin de realiser les operations arithmetiques, nous avons choisi de convertir les vecteurs de bits d'entree en entiers, d'effectuer l'operation, puis de reconverter le resultat vers un vecteur. Le resultat intermediaire est stocke sur 16 bits afin de prendre en compte le cas de la multiplication. En sortie, nous recuperons les 8 bits de poids faible.

Les flags sont mis a jour a partir du resultat sur 16 bits et de l'instruction selectionnee. Les operations logiques sont realisees bit a bit avec une boucle `for`, ce qui reste une construction synthetisable.

L'operation de division n'a pas ete implementee, car il nous avait ete indique en TP qu'elle etait plus delicate a synthetiser directement dans ce contexte.

2.4 Bancs de donnees

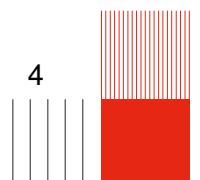
Nous avons choisi de représenter les bancs de données sous forme de tableaux de vecteurs de bits. Ils sont accessibles en lecture et en écriture pour le banc de registres et la mémoire de données, et uniquement en lecture pour la mémoire d'instructions.

- banc de registres : 16 elements de 8 bits ;
- memoire de donnees : 32 elements de 8 bits ;
- memoire d'instructions : 32 elements de 32 bits.

Chacun de ces bancs est synchronise sur le front montant de l'horloge. Ce choix rend le comportement plus previsible en simulation et rapproche le modele de l'implementation materielle.

2.5 Pipeline minimale

Dans le projet Vivado, la pipeline minimale est organisee autour de quatre registres d'etage :



```
LI_DI -> DI_EX -> EX_MEM -> MEM_RE
```

A chaque front d'horloge, l'information avance d'un etage. Notre version implemente surtout l'instruction AFC et COP . Ce choix a permis de valider le principe de propagation dans la pipeline avec les instructions les plus simple, car AFC ne depend pas de l'UAL ni de deux registres source et COP depend seulement du banc de registre.

Cependant, pour aller plus loin, il ne suffit pas de propager l'instruction brute. Il faut aussi propager :

- la destination ;
- le resultat calcule ;
- le signal indiquant s'il faut ecrire dans le banc de registres.

2.6 Decodage des bits de poids faible

Dans la partie compilateur, les instructions sont encodees sur 32 bits avec un opcode sur 8 bits. En pratique, notre jeu d'instructions contient seulement une douzaine d'instructions. Les opcodes utilises restent donc petits et tiennent sur les 4 bits de poids faible de l'octet d'opcode.

Par exemple, l'instruction AFC a pour opcode 6 :

```
opcode complet sur 8 bits : 00000110  
bits de poids faible : 0110
```

C'est pour cette raison que, dans notre version VHDL, nous avons teste seulement :

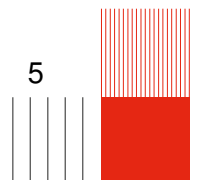
```
LI_DI(27 downto 24) = "0110"
```

au lieu de tester directement l'octet complet :

```
LI_DI(31 downto 24) = "00000110"
```

Ce choix fonctionne tant que le nombre d'instructions reste inferieur a 16, car les 4 bits de poids fort de l'opcode restent a zero. Il simplifie le decodage dans la version minimale. En revanche, si le jeu d'instructions augmente, il faudra decoder l'opcode complet sur 8 bits pour eviter les collisions.

Le meme raisonnement s'applique aux adresses de registres. Les champs de l'instruction sont sur 8 bits, mais le banc de registres ne contient que peu de registres. On utilise donc seulement les bits de poids faible du champ pour adresser le banc de registres.



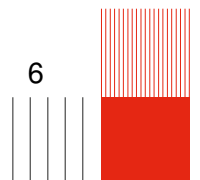
3 LIMITES ET TRAVAIL RESTANT

La principale limite actuelle est que la partie compilateur est plus avancée que la pipeline minimale. Le compilateur est capable de produire plusieurs types d'instructions, alors que la pipeline VHDL minimale exécute surtout AFC et COP.

Pour finir l'intégration, il faudrait :

- généraliser le decodage pour toutes les instructions utiles ;
- propager le résultat calculé, la destination et le signal d'écriture ;
- ajouter le write-back pour ADD, SOU et MUL ;
- gérer les comparaisons et les sauts ;
- ajouter une stratégie simple contre les aléas de données, par exemple des NOP.

Le point important est que les briques principales existent déjà : l'UAL, le banc de registres, la mémoire d'instructions, la mémoire de données et les registres de pipeline. Le travail restant porte surtout sur la logique de contrôle qui relie ces blocs entre eux.

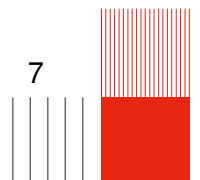


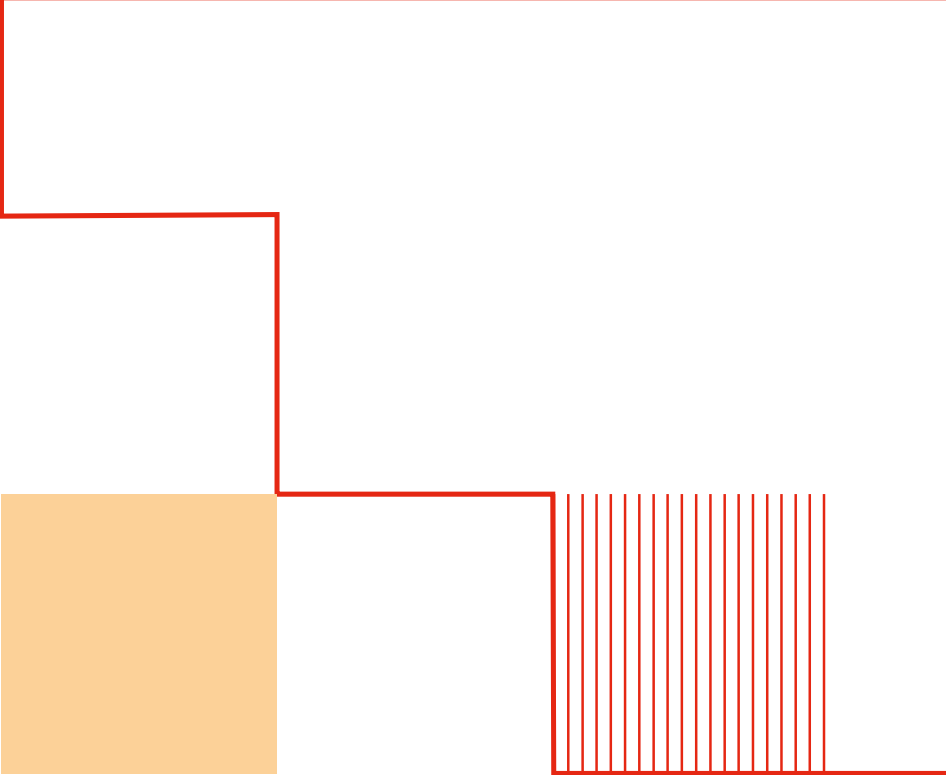
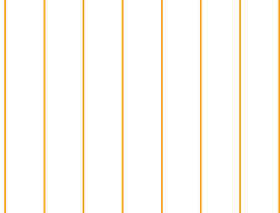
CONCLUSION

Ce projet nous a permis de relier concrètement la compilation d'un sous-ensemble du langage C avec l'exécution sur une architecture matérielle en VHDL. Côté compilateur, nous avons mis en place l'analyse du code, la gestion des variables avec une table des symboles et la génération d'instructions 32 bits.

Côté VHDL, nous avons implémenté les blocs principaux d'un processeur simple : UAL, banc de registres, mémoires et pipeline. La pipeline minimale valide le principe de propagation des instructions avec AFC et COP.

La principale difficulté reste l'intégration complète entre les deux parties : généraliser le decodage, gérer le write-back et traiter les aléas. Le projet constitue donc une base cohérente, qu'il reste à compléter pour exécuter tout le jeu d'instructions généré par le compilateur.





INSA TOULOUSE

135 avenue de Ranguel
31400 Toulouse

Tel : +33 (0)5 61 55 95 13
www.insa-toulouse.fr

